

Variance Reduction Techniques for SGD

Sarmad Ahmed

https://github.com/sarmad-21/sgd_variance_reduction

Abstract

This paper focuses on studying stochastic gradient descent, an algorithm developed particularly for machine learning with large scale amounts of data. Stochastic gradient descent expands upon regular gradient descent; instead of computing the gradient using all data points, the algorithm computes the gradient over a single randomly selected data point in each iteration to estimate the gradient and update the parameters. This differentiation is what makes stochastic gradient descent efficient for large scale problems, but since one point is used to compute the gradient at each iteration, variance is introduced into the gradient, which causes stochastic gradient descent to exhibit a noisy optimization path so it struggles to converge at the minimum. To solve this problem, variance reduction algorithms have been developed for stochastic gradient descent which achieve faster convergence rates than standard stochastic gradient descent while retaining computational efficiency. The variance reduction algorithms for stochastic gradient descent studied in this paper include SAGA, SVRG, and SARAH. All four methods are implemented in Python from scratch and their convergence behavior is evaluated in convex, strongly convex, and non convex settings. Results show that the variance reduction methods converge significantly faster than regular stochastic gradient descent with smaller final gradient norms.

1 Introduction

Optimization is an important concept in machine learning as the main objective in machine learning is finding the best weights that minimize a loss function. Gradient descent is widely used in machine learning, but since it uses all the data points to compute the gradient, in problems with large scale amounts of data it runs for a long time, which is why stochastic gradient descent was developed. Stochastic gradient descent lessens the run time of gradient descent by computing the gradient at one random point at each iteration rather than computing the gradient across all data points and taking the average as it is done in regular gradient descent. The problem with stochastic gradient descent is that since it uses a random data point to compute the gradient during each iteration, variance is introduced to the gradient across each iteration, resulting in variance in the updates for the weights at each iteration as well. The variance problem makes stochastic gradient descent converge slowly and at times not even reach the optimal minimum value. Researchers looked into this problem and developed variance reduction algorithms for stochastic gradient descent to combat this

issue while keeping the benefit of faster run time compared to regular gradient descent. This paper aims to implement, evaluate, and compare the performance of the variance reduction techniques SAGA, SVRG, and SARAH using Python in convex, strongly convex, and non convex settings, using the number of iterations needed to converge, comparing the final optimal values of each algorithm with the actual optimal, and comparing the final gradient norms of each algorithm.

2 Technical Background

2.1 Convexity and Strong Convexity

A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is said to be convex if

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

holds for any two points x, y in the domain of f and $\theta \in [0, 1]$. Geometrically this means that the line segment connecting any two points x and y on the function lies above or on the function. Convex functions are globally lower bounded by a linear function and if a local minimum exists it is guaranteed to be a global minimum.

On the other hand, a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is said to be μ -strongly convex if

$$f(y) \geq f(x) + \nabla f(x)^T (y - x) + \frac{\mu}{2} \|y - x\|^2$$

holds for any two points x, y in the domain of f and $\mu > 0$. Strong convexity means that the function curves upward by at least μ everywhere, guaranteeing a unique global minimum. Strongly convex functions have guaranteed curvature unlike convex functions which may have flat regions near the minimum.

In this paper, convex and strongly convex settings are studied where the convex setting uses logistic regression with the log loss as the cost function. The strongly convex setting adds an L2 regularization term $\frac{\lambda}{2} \|w\|^2$ to the cost function. Since the Hessian of $\frac{\lambda}{2} \|w\|^2$ with respect to w is λI , the objective curves upward by at least $\mu = \lambda$ everywhere, making the problem strongly convex. This paper uses $\lambda = 0.1$ for the strongly convex setting.

2.2 Lipschitz Continuity

A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is said to have an L -Lipschitz continuous gradient if there exists a constant $L > 0$ such that

$$\|\nabla f(x) - \nabla f(y)\| \leq L \|x - y\| \quad \forall x, y \in \mathbb{R}^n$$

For the strongly convex setting (logistic regression with log loss and L2 regularization) the Lipschitz constant is

$$L = \frac{1}{4n} \sum_{i=1}^n \|x_i\|^2 + \lambda$$

For the convex setting (logistic regression with no L2 regularization, $\lambda = 0$), the Lipschitz constant is

$$L = \frac{1}{4n} \sum_{i=1}^n \|x_i\|^2$$

L smooth Lipschitz functions cannot grow faster than quadratic functions and the Lipschitz constant is used as the step size.

2.3 Objective function

The function being minimized in this paper is

$$\min f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w)$$

where n is the number of training samples and each $f_i(w)$ is the loss corresponding to the i th data point. For the logistic regression setting, each $f_i(w)$ is defined as

$$f_i(w) = -y_i \log(\sigma(w^T x_i)) - (1 - y_i) \log(1 - \sigma(w^T x_i)) + \frac{\lambda}{2} \|w\|^2$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function and $y_i \in \{0, 1\}$ is the binary label, and w is the weight vector being optimized.

2.4 Gradient Descent

Gradient descent minimizes the objective function $f(w)$ by computing the gradient with respect to w and evaluating it across all data points and taking the average. The gradient of the objective function is given by

$$\nabla f(w) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w)$$

where for logistic regression the individual gradient for a point is

$$\nabla f_i(w) = (\sigma(w^T x_i) - y_i)x_i + \lambda w$$

The weight is then updated using

$$w_{t+1} = w - t \nabla f(w)$$

where t is the step size which is set to $\frac{1}{L}$ where L is the Lipschitz constant. The step size $t = \frac{1}{L}$ is the step size where maximum descent occurs via the descent lemma.

2.5 Stochastic Gradient Descent

Stochastic gradient descent minimizes the objective function by computing the gradient of the loss function for a single randomly selected point with respect to w and evaluating it at that data point. The gradient of the loss function for a single point is given by

$$\nabla f_i(w) = (\sigma(w^T x_i) - y_i)x_i + \lambda w$$

The weight is then updated using

$$w_{k+1} = w_k - t \nabla f_{i_k}(w_k)$$

where i_k is a randomly selected point at iteration k and t is the step size. Since only one data point is used per iteration the computational cost per iteration is $O(1)$ compared to $O(n)$ for gradient descent which stochastic gradient descent significantly more efficient and faster for large scale datasets which are used often in machine learning. Since a single random point is used to estimate the gradient at each iteration variance is introduced into the gradient estimate. With a fixed step size $t = 1/L$ stochastic gradient descent does not converge to the true minimum so to guarantee convergence a decreasing step size is required. In the strongly convex setting the step size is $t_k = \frac{1}{\mu k}$ giving a convergence rate of $O(1/k)$. In the convex setting the step size is $t_k = \frac{1}{L\sqrt{k}}$ giving a convergence rate of $O(1/\sqrt{k})$. In this paper stochastic gradient descent is evaluated with both a fixed step size $t=1/L$ and a decreasing step size to demonstrate the difference in convergence behavior.

3 Variance Reduction Methods and Implementation

3.1 Stochastic Gradient Descent

Stochastic gradient descent is evaluated in both the convex and strongly convex settings using two different step size rules. A fixed step size rule is used and a decreasing step size rule is used as well. For the fixed step size, t is set to $t = \frac{1}{L}$ where L is the Lipschitz constant as shown above for both settings. In the decreasing step size setting the step size is set to $t_k = \frac{1}{L\sqrt{k}}$ for the convex setting and $t_k = \frac{1}{\mu k}$ for the strongly convex setting where $\mu = \lambda = 0.1$. For both the convex and strongly convex settings w is initialized to 0 and run for 100 epochs. The fixed step size is used to demonstrate the noisy behavior of stochastic gradient descent while the decreasing step size is used to demonstrate that stochastic gradient descent converges with a decreasing step size. The decreasing step size guarantees convergence due to the fact that as the number of iterations increase the step size shrinks which forces the w updates to become smaller as the gradient continues to become noisy.

3.2 SVRG

SVRG (Stochastic Variance Reduced Gradient) reduces the variance of the stochastic gradient by using snapshot weights. Instead of using the gradient evaluated at a single point, SVRG uses the difference between the gradient at the current weights and the gradient at the snapshot weights to reduce the noise of the gradient estimate in stochastic gradient descent. SVRG uses two loops. At the start of each outer loop the current weights w are frozen and saved as the snapshot weights $w_{snapshot}$. The full gradient is then computed at the snapshot weights using all data points which is an expensive computation but only occurs once per outer loop iteration. The inner loop uses a single random data point to compute the gradient

at the current weights and the gradient at the snapshot weights. The variance reduced gradient is then computed as

$$v = \nabla f_i(w) - \nabla f_i(w_{\text{snapshot}}) + \nabla f(w_{\text{snapshot}})$$

where w_{snapshot} are the snapshot weights and $\nabla f(w_{\text{snapshot}})$ is the full gradient computed using all data points at the snapshot weights. The term $\nabla f_i(w) - \nabla f_i(w_{\text{snapshot}})$ reduces the variance as when the current weights get closer to the snapshot weights the difference in their gradients approaches zero reducing the noise in the update. The weights are then updated using

$$w = w - t \cdot v$$

SVRG uses a fixed step size $t = \frac{1}{10L}$ and is initialized with $w = 0$ for both the convex and strongly convex settings and run for 100 epochs.

3.3 SAGA

SAGA is a method that reduces the variance of stochastic gradient descent by using a gradient table. The gradient table stores the most recently computed gradient for each data point. At the start of the algorithm the gradient table is initialized by computing the gradient of the loss function for every data point and the average of all the stored gradients is also computed and stored. At each iteration a random data point is selected and the gradient at the current weights is computed for that point. This new gradient for this point, the old stored gradient for this point, and the average of all the stored gradients are used to compute a new gradient estimate

$$v = \nabla f_i(w) - \text{grad_table}[i] + \text{avg_grad}$$

where $\nabla f_i(w)$ corresponds to the new gradient for the randomly selected point, $\text{grad_table}[i]$ corresponds to the stored gradient for the randomly selected data point, and avg_grad is the average of all the stored gradients. The weights are then updated using

$$w = w - t \cdot v$$

After this update is complete before the new iteration begins the gradient in the table for point i and the average of all the gradients are updated. The term $\nabla f_i(w) - \text{grad_table}[i]$ reduces the variance as near convergence the new gradient for point i approaches the old stored gradient for point i making their difference also approach zero and eliminating noise. SAGA does not require a full gradient computation but instead maintains a gradient table which requires storing as many gradients as there are data points in memory. SAGA uses a fixed step size of $t = \frac{1}{3L}$ and is initialized with $w = 0$ for both the convex and strongly convex settings and run for 100 epochs.

3.4 SARAH

SARAH (Stochastic Recursive Gradient Algorithm) is a variance reduction method that combines SVRG and SAGA. It uses two loops where the full gradient is computed at the start of each outer loop iteration but unlike SVRG it uses a recursive update to estimate the gradient instead of comparing it to the snapshot weights. At the start of each outer loop the current weights are saved as w_0 and the full gradient is computed at w_0 using all data points. This is the initial gradient estimate and the first weight update is made using this full gradient as

$$w = w_0 - t \cdot \nabla f(w_0)$$

The inner loop then performs updates by selecting a random data point i and computing the gradient at the current weights and the previous weights w_{t-1} . The gradient estimate is then updated as

$$v = \nabla f_i(w) - \nabla f_i(w_{t-1}) + v_{prev}$$

where $\nabla f_i(w)$ is the gradient at the current weights for point i , $\nabla f_i(w_{t-1})$ is the gradient at the previous weights for point i , and v_{prev} is the previous gradient estimate. The weights are then updated using

$$w = w - t \cdot v$$

The SARAH algorithm has each gradient estimate build on the previous one by incorporating the change in gradient between the current weights and previous weights. As the algorithm converges the current and previous weights get close to each other so the difference in their gradients approaches zero so the gradient estimate becomes more accurate and noise is reduced. SARAH uses a fixed step size of $t = \frac{1}{L}$ and is initialized with $w = 0$ for both the convex and strongly convex settings and run for 100 epochs.

3.5 Brief Exploration of Non Convex Setting

This paper briefly explores the behavior of stochastic gradient descent and SAGA in a non convex setting. Both methods are evaluated on a neural network for binary classification with two layers. The neural network is made up of a hidden layer with 50 input features, 32 neurons followed by a ReLU activation function, and an output layer with sigmoid activation. The objective function is the binary cross entropy loss function but unlike the logistic regression setting the neural network objective is non convex due to the usage of ReLU, a nonlinear activation function. Since this is a non convex problem convergence to a global minimum is not guaranteed and the true optimal value cannot be computed. Convergence is evaluated using the training loss and gradient norm only. Both methods use a fixed step size of $t = 0.01$ and are run for 50 epochs. Initially the goal was to explore SARAH in a non convex setting but it was abandoned due to the high computational cost of SARAH's inner loop while SAGA only requires a single point gradient computation per iteration which requires less computational cost than SARAH.

4 Numerical Results

4.1 Setup

All experiments are run on a synthetically generated dataset using scikit-learn with 5000 samples and 50 features. The true optimal cost value for each setting is computed using Newton’s method via scipy’s `minimize` function with the Newton CG solver. All methods are initialized with $w = 0$ and run for 100 epochs. Convergence is defined as when the optimality gap $f(w) - f(w^*)$ drops below a threshold of $\varepsilon = 0.001$.

4.2 Convex Setting Results

The true optimal cost for the convex setting computed using Newton’s method is $f(w^*) = 0.1303$. In the convex setting stochastic gradient descent with fixed step size does not converge in 100 epochs and the cost bounces around confirming that stochastic gradient descent with fixed step size does not converge. It achieves a final cost of 0.1799 with an optimality gap of 0.0496 and a final gradient norm of 0.0559. Stochastic gradient descent with decreasing step size converges in 55 epochs achieving a final cost of 0.1308 with an optimality gap of 0.00046 and a final gradient norm of 0.0043. SVRG converges the fastest taking 2 epochs achieving a final cost of 0.1303, an optimality gap of -2.07×10^{-14} , and a final gradient norm of 3.25×10^{-14} . SAGA takes 4 epochs to converge with a final cost of 0.1303, an optimality gap of -2.05×10^{-14} , and a final gradient norm of 1.72×10^{-15} . Finally SARAH takes 12 epochs to converge with a final cost of 0.1303, an optimality gap of -2.06×10^{-14} , and a final gradient norm of 2.44×10^{-14} . All variance reduction methods converge to the true optimal. SVRG converges the fastest in the convex setting. Figures showing cost vs epochs, gradient norm vs epochs, and optimality gap vs epochs are shown below.

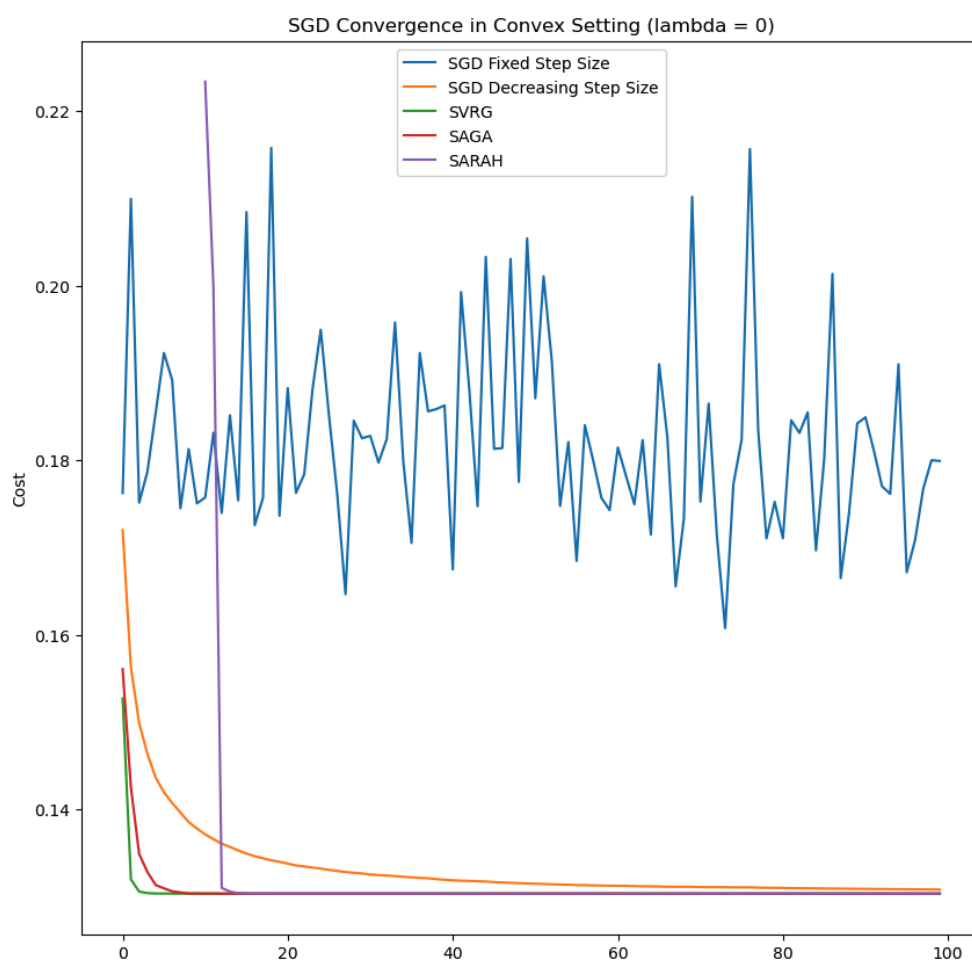


Figure 1: Cost vs Epochs (Convex Setting)

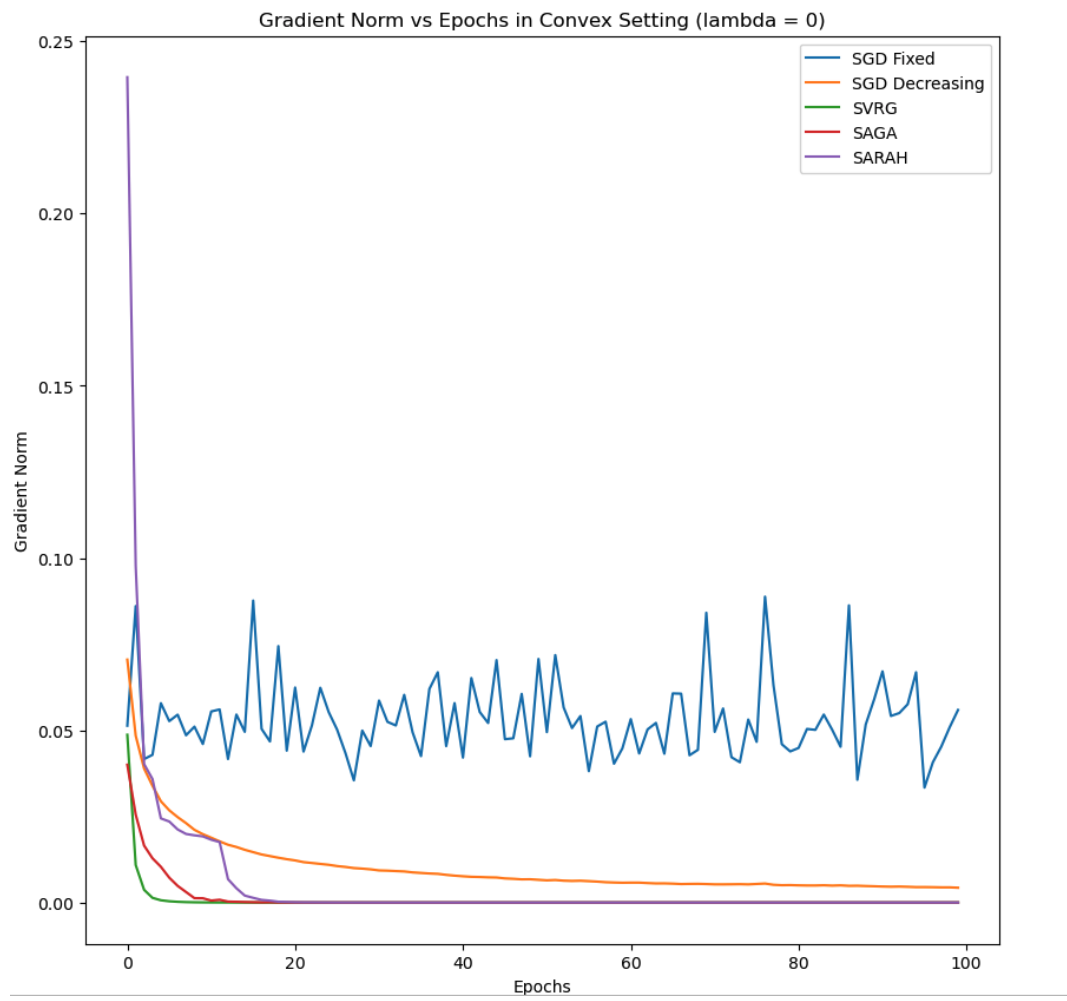


Figure 2: Gradient Norm vs Epochs (Convex Setting)

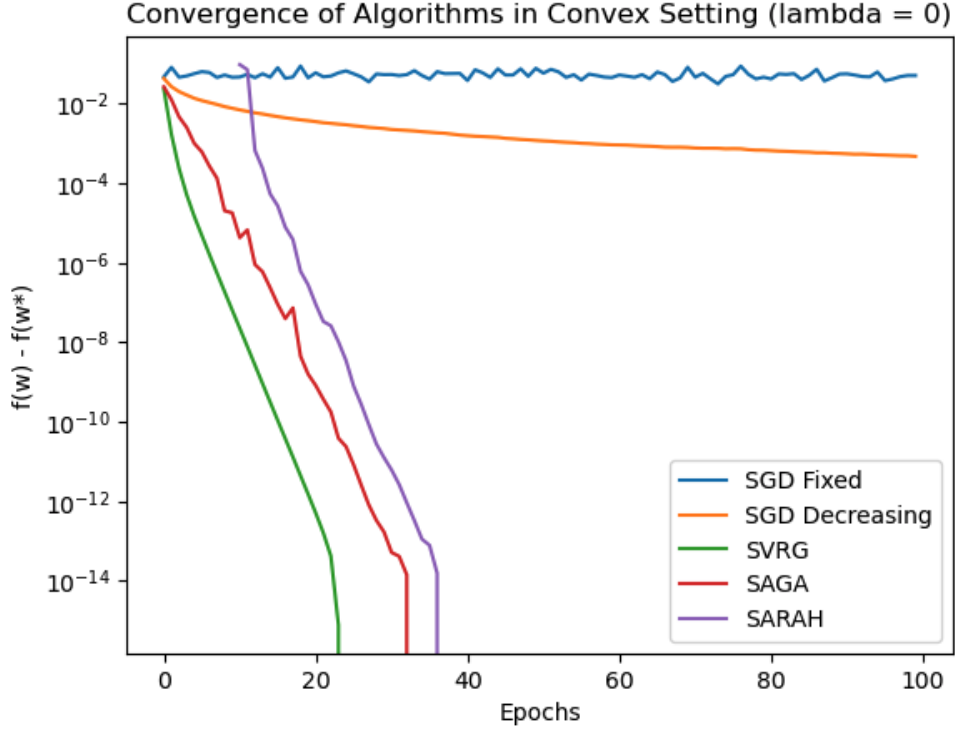


Figure 3: Optimality Gap vs Epochs (Convex Setting)

4.3 Strongly Convex Setting Results

The true optimal cost for the strongly convex setting computed using Newton's method is $f(w^*) = 0.3188$. In the strongly convex setting stochastic gradient descent with fixed step size does not converge in 100 epochs achieving a final cost of 0.3796 with an optimality gap of 0.0608 and a final gradient norm of 0.1663. Stochastic gradient descent with decreasing step size converges in 1 epoch achieving a final cost of 0.3188 with an optimality gap of 9.03×10^{-6} and a final gradient norm of 0.0020. SVRG converges the fastest taking 1 epoch achieving a final cost of 0.3188, an optimality gap of -2.39×10^{-15} , and a final gradient norm of 7.68×10^{-16} . SAGA takes 3 epochs to converge with a final cost of 0.3188, an optimality gap of -2.28×10^{-15} , and a final gradient norm of 2.83×10^{-15} . Finally SARAH takes 5 epochs to converge with a final cost of 0.3188, an optimality gap of -2.28×10^{-15} , and a final gradient norm of 3.55×10^{-14} . All variance reduction methods converge to the true optimal. The strongly convex setting shows faster convergence than the convex setting for all methods which is consistent with theory as strong convexity provides additional curvature that accelerates convergence. Figures showing cost vs epochs, gradient norm vs epochs, and optimality gap vs epochs are shown below.

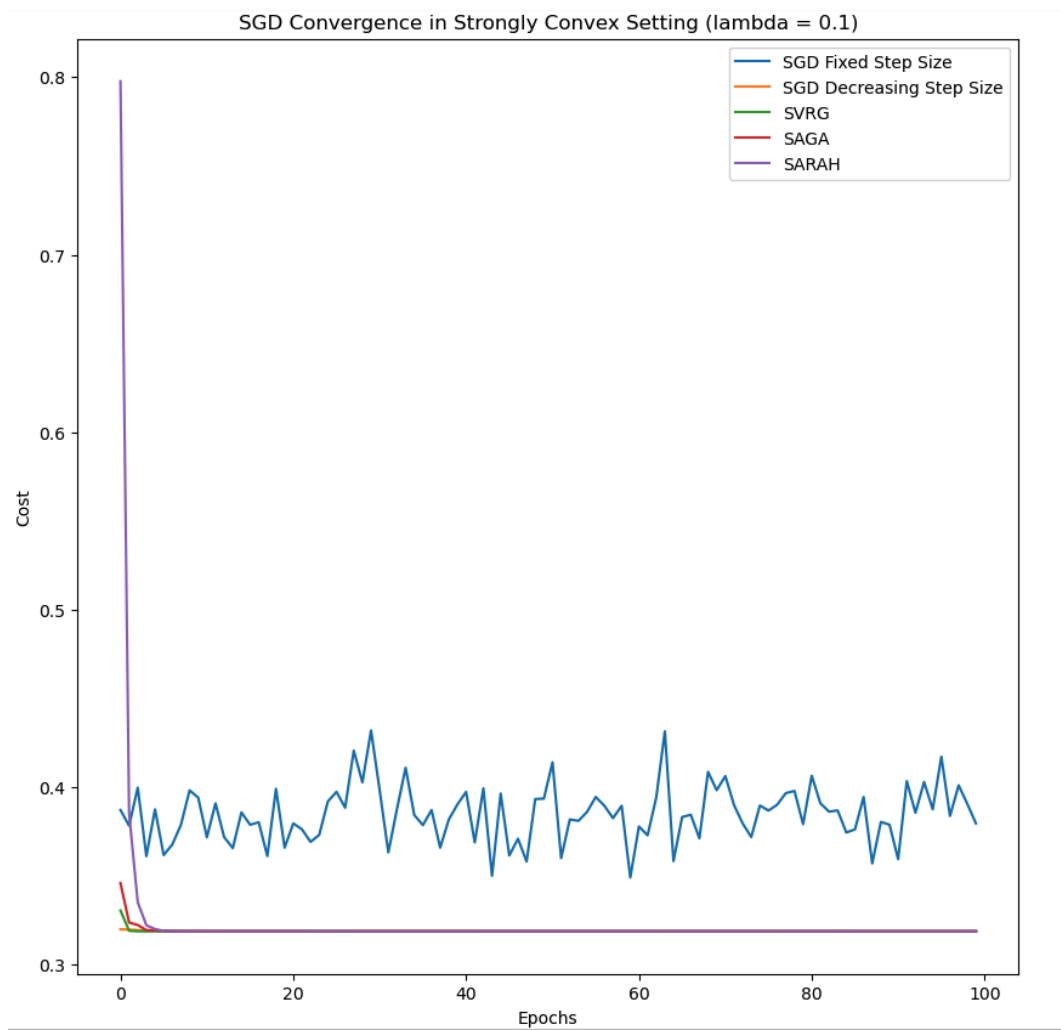


Figure 4: Cost vs Epochs (Strongly Convex Setting)

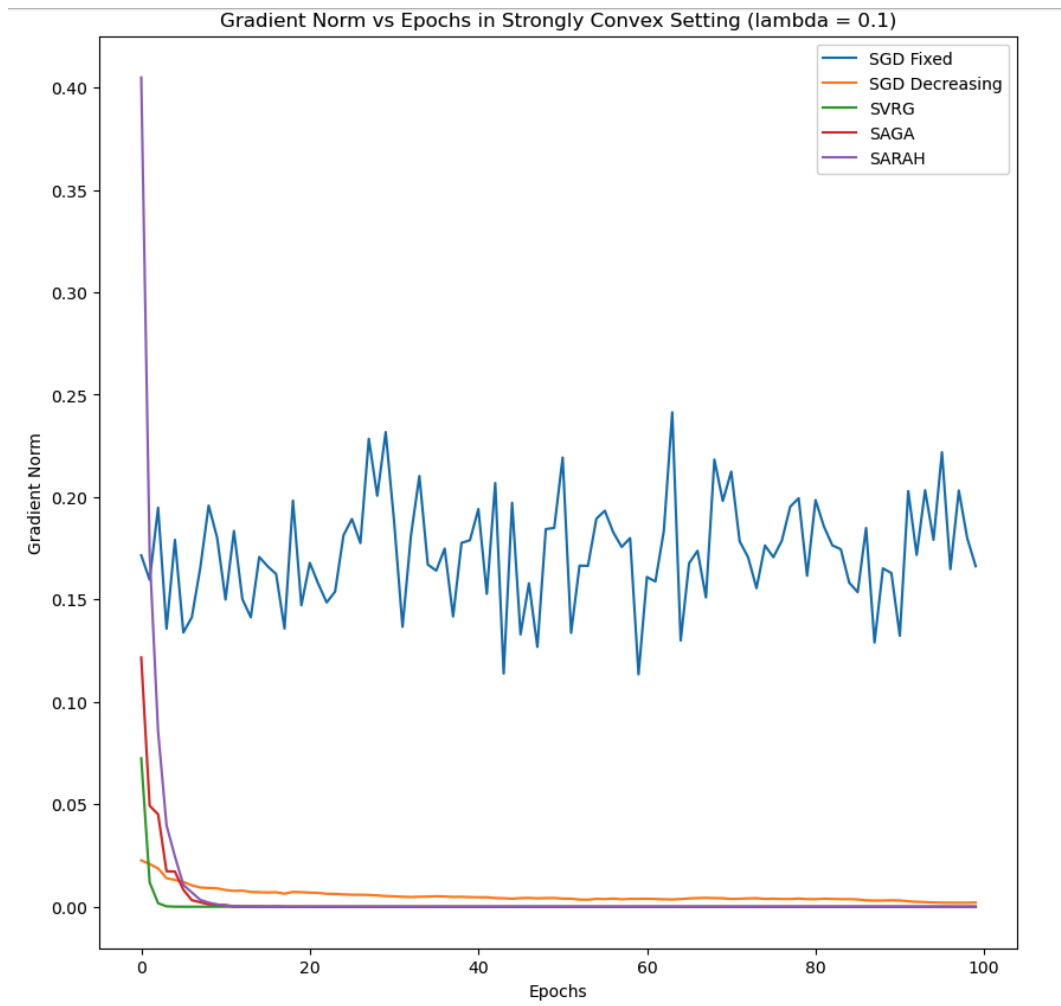


Figure 5: Gradient Norm vs Epochs (Strongly Convex Setting)

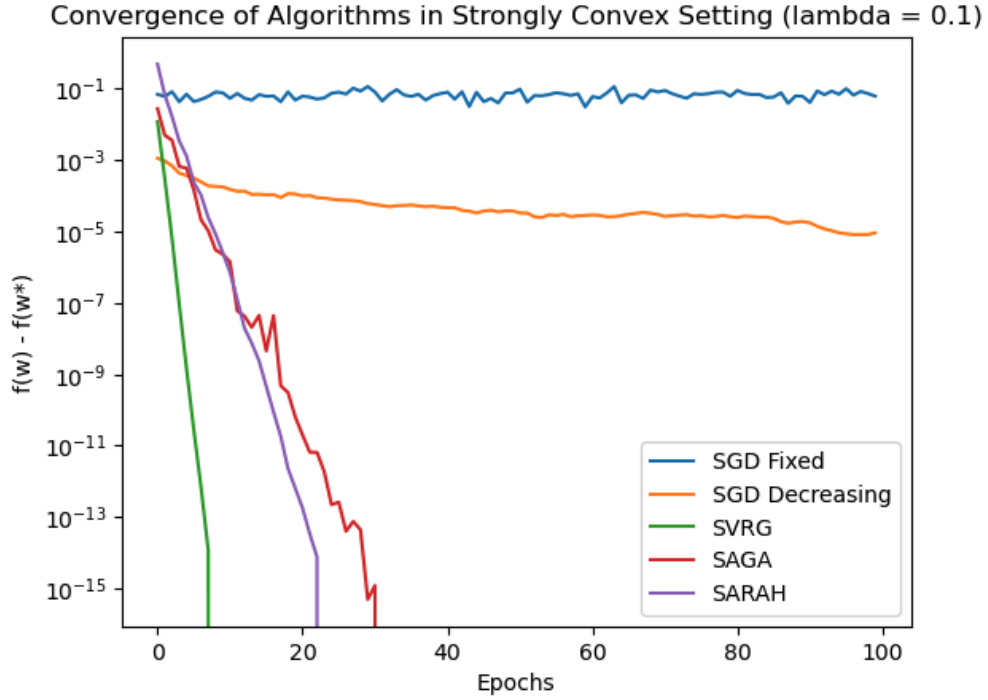


Figure 6: Optimality Gap vs Epochs (Strongly Convex Setting)

4.4 Non Convex Setting Results

In the non convex neural network setting both stochastic gradient descent and SAGA are evaluated and compared. Since this is a non convex problem there is no true optimal value to compare the optimal values found by the algorithms to so only the training loss and gradient norm are reported. SGD achieves a final training loss of 0.0011 and a final gradient norm of 0.0021 after 50 epochs. SAGA achieves a slightly lower final training loss of 0.0009 and a final gradient norm of 0.0019 after 50 epochs. SAGA converges to a slightly better optimal value than SGD confirming that variance reduction algorithms perform better even in non convex settings. Figures showing training loss vs epochs and gradient norm vs epochs are shown below.

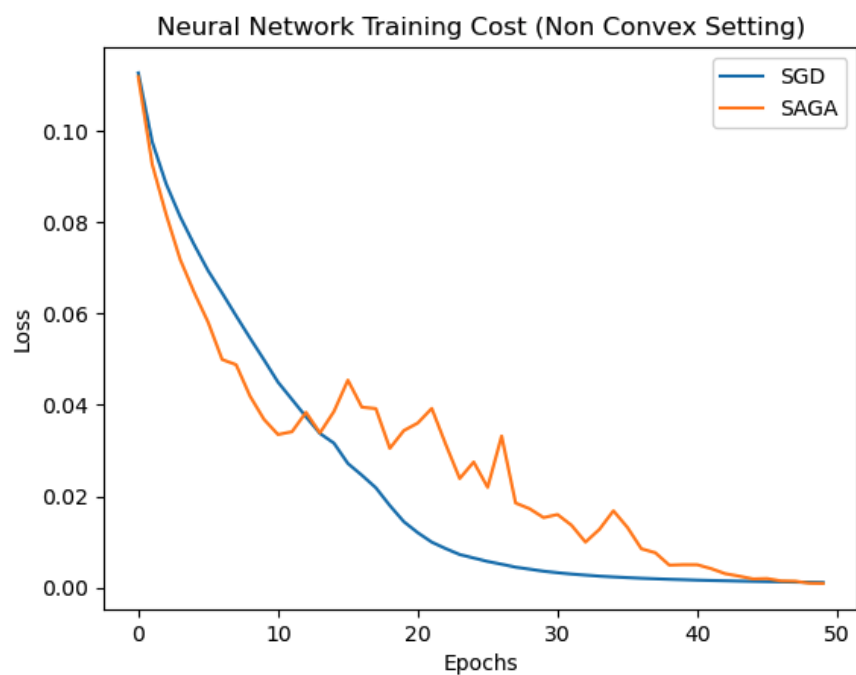


Figure 7: Training Loss vs Epochs (Non Convex Setting)

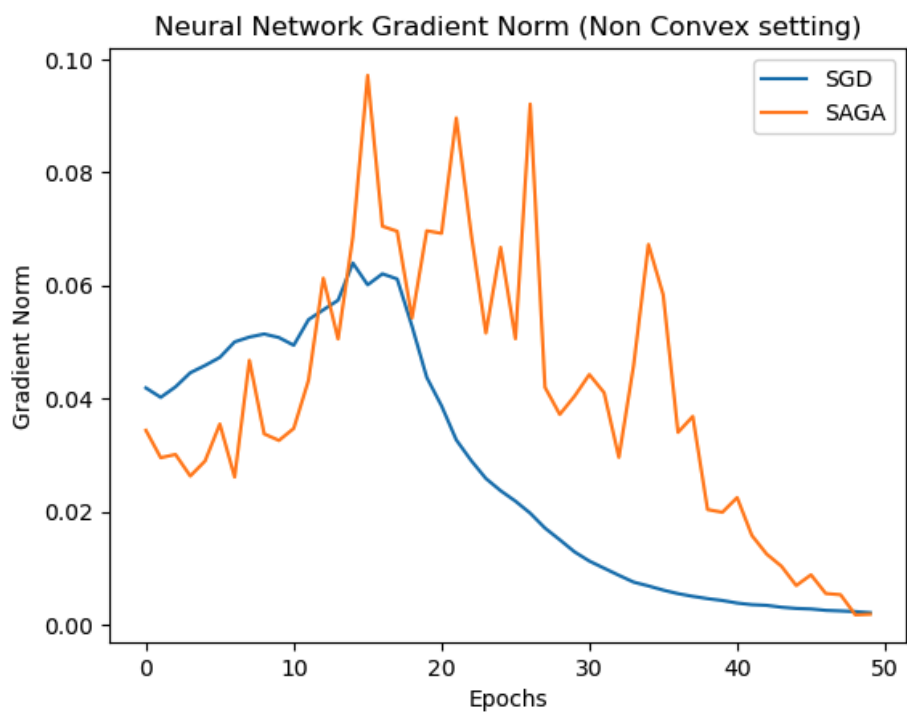


Figure 8: Gradient Norm vs Epochs (Non Convex Setting)

5 Conclusion

This paper implemented and evaluated stochastic gradient descent and three variance reduction methods including SAGA, SVRG, and SARAH. These algorithms were evaluated and compared in convex, strongly convex, and non convex settings. The results show that variance reduction methods converge faster than stochastic gradient descent with fixed step size and achieve final gradient norms approximately equal to 0. Stochastic gradient descent with fixed step size fails to converge in both the convex and strongly convex settings while the variance reduction methods converge to the true optimal cost value in as few as 1-12 epochs. The strongly convex setting shows faster convergence than the convex setting for all methods due to the guaranteed curvature. In the non convex neural network setting both stochastic gradient descent and SAGA converge with SAGA achieving a slightly lower final training loss confirming that variance reduction perform better even in non convex settings. Overall, this paper explores variance reduction techniques in different settings and discusses the theory behind the different algorithms.

References

- [1] R. M. Gower, M. Schmidt, F. Bach, and P. Richtárik, “Variance-Reduced Methods for Machine Learning,” *Proceedings of the IEEE*, vol. 108, no. 11, pp. 1968–1983, 2020.
- [2] L. M. Nguyen, J. Liu, K. Scheinberg, and M. Takáč, “SARAH: A Novel Method for Machine Learning Problems Using Stochastic Recursive Gradient,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.
- [3] L. M. Nguyen, M. van Dijk, D. T. Phan, P. H. Nguyen, T. W. Weng, and J. R. H. Moulin, “Inexact SARAH Algorithm for Stochastic Optimization,” *arXiv preprint arXiv:1811.10105*, 2018.
- [4] R. Johnson and T. Zhang, “Accelerating Stochastic Gradient Descent Using Predictive Variance Reduction,” in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 315–323, 2013.
- [5] H. Behl, “Stochastic Variance Reduced Gradient,” lecture notes, 2019. [Online]. Available: <https://harkiratbehl.github.io/projects/svrg/svrg.pdf>
- [6] P. Virtanen et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.